

# Day 8: Object detection at the edge

Naeemullah Khan

[naeemullah.khan@kaust.edu.sa](mailto:naeemullah.khan@kaust.edu.sa)



جامعة الملك عبد الله  
للعلوم والتقنية  
King Abdullah University of  
Science and Technology

KAUST Academy  
King Abdullah University of Science and Technology

October 7, 2026

- ▶ Days 5 and 7 gave one label for a complete image. A detector answers two questions for each object: which class it has, and where it is. Its output is a list of boxes, each with a class and a score.
- ▶ A single-stage detector gives thousands of candidate boxes for one image. The post-processing removes the duplicates. This step runs on the processor after the model, and it is a part of the latency.
- ▶ One accuracy value does not describe a detector. IoU, precision, recall, and mAP say how good the boxes are. The input resolution changes the frame rate and the smallest object that the model finds.
- ▶ In the lab you run an SSD model and a YOLO model on the Raspberry Pi, train a YOLO model on a small custom dataset, export it to NCNN and to LiteRT in int8, and measure the frame rate with the camera.

After this day you can:

- ▶ Describe the structure of a single-stage detector.
- ▶ Compute IoU, apply non-maximum suppression, and read an mAP result.
- ▶ Train a custom YOLO model and export it for the Raspberry Pi.
- ▶ Tune input resolution and precision for a target frame rate.

## Part 1 Detection models

Exercise: apply non-maximum suppression to six boxes by hand

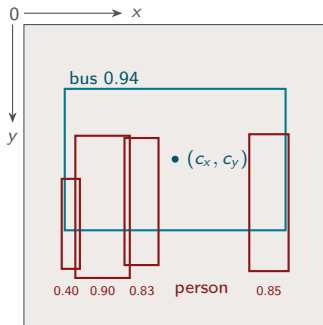
## Part 2 Metrics and data

Exercise: calculate the IoU of two box pairs

## Part 3 Deployment

Exercise: predict the frame rate when the resolution goes from 640 to 320

# A detection: a class, a score, and a box



## Box format

## Four numbers

Two corners

$x_1, y_1, x_2, y_2$

Centre and size

$c_x, c_y, w, h$

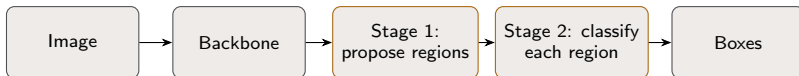
- ▶ The SSD file of the lab gives  $y_1, x_1, y_2, x_2$  from 0 to 1. The raw output of YOLO gives  $c_x, c_y, w, h$  in pixels of the model input.

**Key point:** Check the box format of each model. A wrong format gives boxes at wrong places, and no error message.

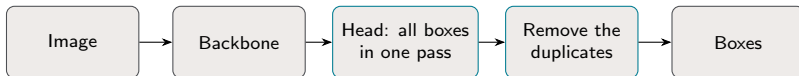
Source: Boxes: YOLO11n on the image bus.jpg of the Ultralytics package, an experiment of this course.

# Two families of detectors

## Two stages



## One stage



- ▶ **Two stages:** R-CNN, Fast R-CNN, Faster R-CNN. The second stage runs for each proposed region. This costs time.
- ▶ **One stage:** SSD, EfficientDet, YOLO, and FOMO. One pass of one network predicts the boxes and the classes for all positions.
- ▶ The cost of a single-stage model does not depend on the number of objects in the image.

**Key point:** The detectors of this course are single-stage detectors. They are the usual choice for an edge device.

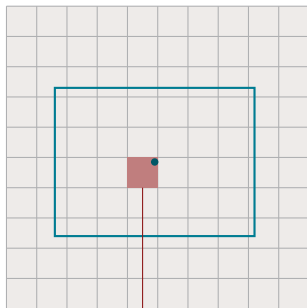
# The parts of a single-stage detector



| Part            | Job                                                                           | Examples                                 |
|-----------------|-------------------------------------------------------------------------------|------------------------------------------|
| Backbone        | It makes feature maps from the image. It is a classifier with no last layers. | MobileNet, EfficientNet                  |
| Neck            | It mixes maps of different depths.                                            | FPN, BiFPN                               |
| Head            | It predicts box values and class scores for each cell of a map.               | Convolutions with shared weights         |
| Post-processing | It makes the final list from the candidates. It has no weights.               | Score threshold, non-maximum suppression |

**Key point:** A new detector changes one of the four parts. Ask which part changed.

# The grid: one prediction for each cell

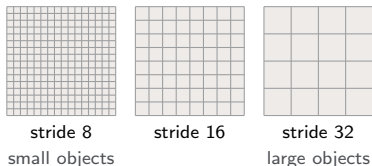


|       |       |     |     |       |       |         |          |
|-------|-------|-----|-----|-------|-------|---------|----------|
| $c_x$ | $c_y$ | $w$ | $h$ | $s_1$ | $s_2$ | $\dots$ | $s_{80}$ |
|-------|-------|-----|-----|-------|-------|---------|----------|

- ▶ The last map of the backbone has  $20 \times 20$  cells for an image of  $640 \times 640$  pixels. One cell covers  $32 \times 32$  pixels: the **stride** is 32.
- ▶ The head gives one vector for each cell: 4 box values and one score for each class.
- ▶ The cell at the centre of an object predicts the box of the complete object. The box can be much larger than the cell.
- ▶ The head is a convolution. It uses the same weights for each cell, as FOMO on Day 5.

**Key point:** A detector is a classifier for each cell of a map, plus four numbers for the box.

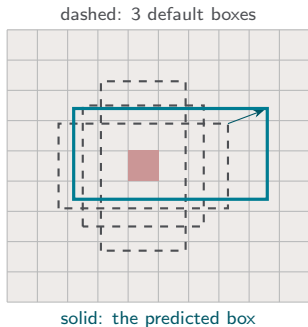
# Three maps for three sizes of objects



| Stride | Map at 640     | Cells |
|--------|----------------|-------|
| 8      | $80 \times 80$ | 6400  |
| 16     | $40 \times 40$ | 1600  |
| 32     | $20 \times 20$ | 400   |
| Sum    |                | 8400  |

- ▶ A deep map knows what the object is. Its cells are coarse.
- ▶ An early map has fine cells. It knows less about the meaning.
- ▶ The neck mixes the maps: each map gets the meaning and the position.
- ▶ The head runs on each map.
- ▶ In the test image, the 10 candidates for the bus come from the map with the stride 32. The small person at the border comes from the map with the stride 16.

Source: YOLO11n on the image bus.jpg, an experiment of this course. The grids show 16, 8, and 4 cells on a side, not the real number.

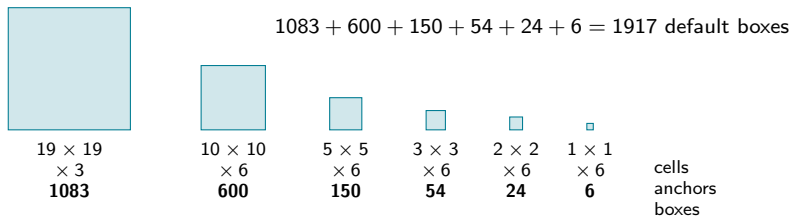


- ▶ An **anchor** is a default box with a fixed size and a fixed shape. Each cell has  $K$  anchors.
- ▶ For each anchor, the head predicts 4 corrections and the class scores:  
$$K \times (4 + \text{classes}) \text{ values for a cell}$$
- ▶ The corrections move the anchor and change its size.
- ▶ The designer selects the sizes and the shapes of the anchors for the objects of the dataset.

- ▶ SSD and EfficientDet use anchors. YOLOv8 and the newer YOLO models use no anchors: each cell predicts the distances to the four sides of the box.

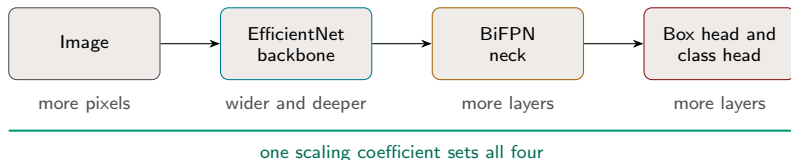
**Key point:** With anchors, the number of candidates is the number of cells times  $K$ .

# SSD: boxes from six maps



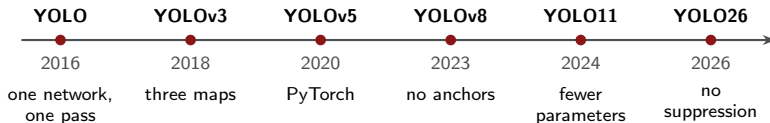
- ▶ SSD: “Single Shot MultiBox Detector” (2016). The backbone of the lab model is MobileNetV1.
- ▶ The input is  $300 \times 300 \times 3$  in uint8. The file has 4 183 312 bytes.
- ▶ For each default box, the head gives 4 box values and 91 class values: 90 classes and the background.
- ▶ The last operator of the file, `TFLite_Detection_PostProcess`, does the post-processing. The outputs are 10 boxes, 10 classes, 10 scores, and the number of valid detections.

Source: The file `ssd-mobilenet-v1-tflite-default-v1.tflite` of the kit lab, read with LiteRT.  
Method: Liu et al., 2016.



- ▶ **BiFPN:** a neck that sends features from the deep maps to the early maps, and back. It learns a weight for each input.
- ▶ **Compound scaling:** the image size, the backbone, the neck, and the heads grow together. This gives a family of models from small to large.
- ▶ The lab file is EfficientDet-Lite0: an input of  $320 \times 320$ , five maps with 40, 20, 10, 5, and 3 cells on a side, and 9 anchors for each cell: 19 206 default boxes.
- ▶ The file has 4 563 519 bytes. It gives 25 boxes.

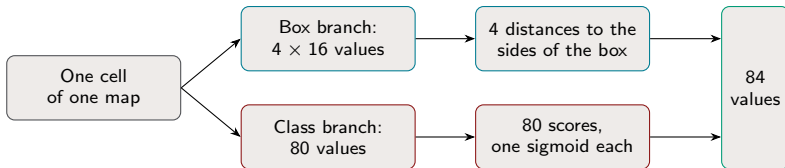
Source: Method: Tan, Pang, and Le, 2020. The EfficientDet-Lite0 file of the kit lab, read with LiteRT.



- ▶ YOLO: “You Only Look Once”. The first model showed that one pass of one network is fast enough for video.
- ▶ Different teams made the versions. A larger number is not always the better model for your task.
- ▶ Each version has five scales: n, s, m, l, and x. YOLO11n has 2.6 million parameters, and YOLO11x has 57.0 million. Part 3 compares the scales.
- ▶ The Ultralytics package trains, exports, and runs the models with one tool. The lab uses it.
- ▶ The package and its models have the licence AGPL-3.0. Read the licence before you use them in a product.

Source: Redmon et al., 2016 and 2018. The Ultralytics package, version 8.4.171. *Edge AI Engineering*, “Computer Vision Applications with YOLO”.

# The head of YOLO11: no anchors



- ▶ **Box:** for each of the four sides, 16 values give a probability for each distance from 0 to 15 cells. The mean is the distance. The program converts the distances to  $c_x$ ,  $c_y$ ,  $w$ ,  $h$ .
- ▶ **Class:** each class has its own sigmoid. The 80 scores of a cell do not add up to 1.
- ▶ The head has no separate score for “an object is here”. The score of a candidate is its largest class score.
- ▶ YOLOv8 has the same head. YOLO26 predicts each distance with one value.

Source: The head of the Ultralytics models, read from the package. The raw outputs have 64 and 80 channels for each cell.

```
from ultralytics import YOLO

model = YOLO("yolo11n.pt")
raw = model.model(image)[0][0]
print(raw.shape)

boxes = raw[:4].T
scores = raw[4:].max(0).values
labels = raw[4:].argmax(0)
keep = scores > 0.25
print(int(keep.sum()))
```

*# image: 1 x 3 x 640 x 640, values 0 to 1*  
*# torch.Size([84, 8400])*  
*# 8400 rows: cx, cy, w, h in pixels*  
*# the largest class score of each cell*  
*# the class with this score*  
*# 47*

- ▶ The output has 84 rows and 8400 columns: 705 600 numbers. The 80 classes are the classes of the dataset COCO.
- ▶ For the test image, 47 cells have a score above 0.25. The image shows 5 objects.
- ▶ Normally, `model.predict()` does these steps and the post-processing for you.

**Key point:** The output of the network is not a list of objects. It is one candidate for each cell.

# How many candidates?

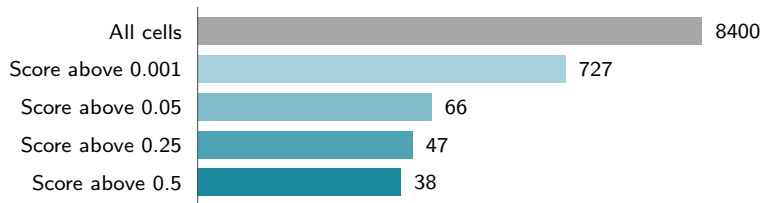
| Model              | Image | Maps | Candidates | Values for each |
|--------------------|-------|------|------------|-----------------|
| SSD MobileNet V1   | 300   | 6    | 1917       | 4 + 91          |
| EfficientDet-Lite0 | 320   | 5    | 19 206     | 4 + 90          |
| YOLO11n            | 320   | 3    | 2100       | 4 + 80          |
| YOLO11n            | 640   | 3    | 8400       | 4 + 80          |
| FOMO               | 160   | 1    | 400        | 3, and no box   |

- ▶ The number of candidates is fixed. It comes from the image size, the strides, and the anchors.
- ▶ An empty image and a full image give the same number.
- ▶ Two times the image side gives four times the candidates.
- ▶ The head of FOMO gives  $20 \times 20$  cells with three classes: the background and two objects.

**Key point:** The model always gives all candidates. The post-processing selects the detections.

Source: An experiment of this course with the model files.

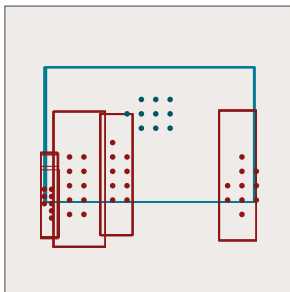
# Step 1: the score threshold



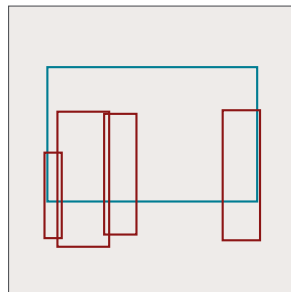
- ▶ YOLO11n on the test image at 640 pixels. The bars have a logarithmic scale.
- ▶ The threshold removes almost all candidates. The package uses 0.25 for a prediction.
- ▶ A high threshold loses objects. The fifth person has the score 0.40: the threshold 0.5 removes this person.
- ▶ A low threshold keeps false boxes, and it gives more work to the next step.

Source: An experiment of this course.

# Measured: 47 candidates for 5 objects



47 boxes, and the cell of each box

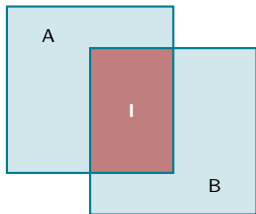


5 boxes after the suppression

- ▶ 10 cells predict the bus, and 37 cells predict the 4 persons.
- ▶ The cells near the centre of an object give almost the same box.
- ▶ These boxes are not errors of the model. The training asks many cells to find each object.

Source: YOLO11n on bus.jpg at 640 pixels, score above 0.25. An experiment of this course.

# IoU: the overlap of two boxes



$$\text{IoU} = \frac{\text{area of the intersection}}{\text{area of the union}}$$

- ▶ IoU: “intersection over union”.
- ▶ The intersection is the area that the two boxes share. The union is the area that one box or the two boxes cover.
- ▶ Here A and B have  $4 \times 4 = 16$  squares each. The intersection has  $2 \times 3 = 6$  squares.
- ▶ The union has  $16 + 16 - 6 = 26$  squares. The IoU is  $6/26 = 0.23$ .

**IoU = 0**

The boxes do not touch.

**IoU = 0.5**

Two equal boxes share two thirds of their area.

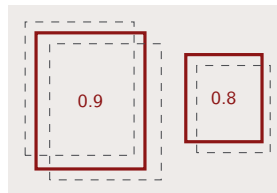
**IoU = 1**

The boxes are the same.

**Key point:** Two candidates with a high IoU show the same object. Part 2 uses the IoU to compare a prediction with a label.

## Step 2: non-maximum suppression

1. Remove the candidates below the score threshold. Sort the other candidates by their score.
2. Take the candidate with the highest score.  
**Keep it.**
3. Calculate its IoU with each other candidate of the same class.
4. **Remove** each candidate with an IoU above the IoU threshold.
5. Do steps 2 to 4 again with the candidates that are left, until no candidate is left.
  - ▶ The method is greedy. It never looks at a decision again.
  - ▶ It has no weights. It is normal code after the network.
  - ▶ For  $N$  candidates, it calculates up to  $N^2/2$  values of the IoU.



**Key point:** The suppression keeps the best box of each group.

| Threshold | Too low                                              | Too high                                  |
|-----------|------------------------------------------------------|-------------------------------------------|
| Score     | False boxes stay. The suppression has more work.     | The model loses objects with a low score. |
| IoU       | Two real objects that are near become one detection. | Two boxes of one object stay.             |

- ▶ The test image with the IoU thresholds 0.3, 0.5, and 0.7: 5 boxes each time. With 0.9: 8 boxes for 5 objects.
- ▶ The package uses 0.25 for the score and 0.7 for the IoU. The arguments are `conf` and `iou`.
- ▶ `agnostic_nms=True` compares boxes of different classes also. Use it when one object gets two classes.
- ▶ A crowd needs a higher IoU threshold than objects that stand alone.

**Key point:** The two thresholds are parameters of the product, not of the model. Tune them on images of your application.

Source: An experiment of this course with YOLO11n.

# Where the suppression runs, and its cost

| Model                                 | Where                                    | Thresholds                                                  |
|---------------------------------------|------------------------------------------|-------------------------------------------------------------|
| SSD and EfficientDet files of the lab | In the file, as its last operator        | Fixed in the file: IoU 0.6 and 0.5, at most 10 and 25 boxes |
| YOLOv8 and YOLO11                     | In the Python program, after the runtime | Arguments of each call                                      |

- ▶ On an x86 CPU, YOLO11n needs 66.7 ms for the network and 1.4 ms for the suppression of 47 candidates.
- ▶ With the score threshold 0.001, 727 candidates go to the suppression: 43 ms. A validation run uses this low threshold.
- ▶ Published for a Raspberry Pi Zero 2 W, YOLO11n in NCNN: 2748 ms for the network and 533 ms for the post-processing.

**Key point:** An exported YOLO network does not include this code. Measure the complete pipeline.

Source: An experiment of this course on an x86 processor. Raspberry Pi: *Edge AI Engineering*, chapter on YOLO.

|             | One-to-many head             | One-to-one head             |
|-------------|------------------------------|-----------------------------|
| Training    | Many cells learn each object | One cell learns each object |
| Output      | Candidates with duplicates   | The final list              |
| Suppression | Necessary                    | Not necessary               |
| Models      | YOLOv8, YOLO11               | YOLO26                      |

- ▶ YOLO26n has the two heads. Its one-to-one head gives 300 rows: a box, a score, and a class. For the test image, 5 rows have a score above 0.25: the same 5 objects.
- ▶ Published for a Raspberry Pi Zero 2 W in PyTorch: 36 ms of post-processing with the one-to-one head, and 192 ms with the one-to-many head.
- ▶ The export to NCNN keeps only the one-to-many head. The export to ONNX keeps the one-to-one head.

**Key point:** Select the model and the export format together.

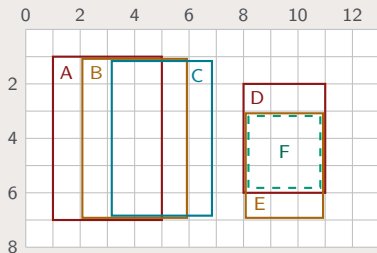
Source: *Edge AI Engineering*, chapter on YOLO.  
Output of YOLO26n: an experiment of this course.

|             | <b>SSD MobileNet V1</b> | <b>EfficientDet-Lite0</b> | <b>YOLO11n</b> | <b>FOMO</b>      |
|-------------|-------------------------|---------------------------|----------------|------------------|
| Image side  | 300                     | 320                       | 640 or 320     | 160              |
| Box         | Anchors                 | Anchors                   | No anchors     | No box: a centre |
| Candidates  | 1917                    | 19 206                    | 8400 or 2100   | 400 cells        |
| Classes     | 90                      | 90                        | 80             | 2                |
| File        | 3.99 MB                 | 4.35 MB                   | 5.35 MB        | 55.3 KB          |
| Weights     | 8 bits                  | 8 bits                    | float16        | 8 bits           |
| Suppression | In the file             | In the file               | In the program | None             |

- ▶ The first three models know the classes of COCO. FOMO is the model of the kit lab for two objects: a box and a wheel.
- ▶ YOLO11n has 2616 248 parameters and 6.5 GFLOP at 640 pixels: about 11 times the MACs of MobileNetV2 at 224 pixels.
- ▶ In the lab, Part A runs the SSD model and a YOLO model on test images. Parts B and C train and export a YOLO model.

Source: An experiment of this course. 1 MB is 1024 KB.

# Exercise: suppression of six boxes by hand



|   | Class  | Score | $x_1, y_1, x_2, y_2$ |
|---|--------|-------|----------------------|
| A | person | 0.90  | 1, 1, 5, 7           |
| B | person | 0.75  | 2, 1, 6, 7           |
| C | person | 0.60  | 3, 1, 7, 7           |
| D | person | 0.85  | 8, 2, 11, 6          |
| E | person | 0.70  | 8, 3, 11, 7          |
| F | dog    | 0.55  | 8, 3, 11, 6          |

A detector gives these six candidates. All scores are above the score threshold. The drawing moves some lines a little, so that you see each box.

1. Apply the suppression for each class, with the IoU threshold 0.5. Write the IoU values that you need. Which boxes stay?
2. Which boxes stay with the IoU threshold 0.3?
3. Which boxes stay with the threshold 0.5, when the suppression compares boxes of all classes?

# Answer: suppression of six boxes by hand

| Pair    | Intersection      | Union               | IoU  | With the threshold 0.5 |
|---------|-------------------|---------------------|------|------------------------|
| A and B | $3 \times 6 = 18$ | $24 + 24 - 18 = 30$ | 0.60 | A removes B            |
| A and C | $2 \times 6 = 12$ | $24 + 24 - 12 = 36$ | 0.33 | C stays                |
| D and E | $3 \times 3 = 9$  | $12 + 12 - 9 = 15$  | 0.60 | D removes E            |
| D and F | $3 \times 3 = 9$  | $12 + 9 - 9 = 12$   | 0.75 | Different classes      |

1. The order is A, D, B, E, C for the persons. A stays and removes B. D stays and removes E. C stays: its IoU with A is 0.33, and nobody compares it with B, because B is not in the list now. F is the only dog. **Result: A, C, D, F.**
  2. The IoU of A and C is 0.33, above 0.3. A removes C also. **Result: A, D, F.**
  3. D removes F: the IoU is 0.75. **Result: A, C, D.**
- ▶ C is a second person next to the first person, or a third box of the same person. The suppression cannot know this. The IoU threshold decides.
  - ▶ F is a dog in front of a person, or a wrong class for the person. With one class for each object in your application, use the suppression for all classes.

## Part 1 Detection models

Exercise: apply non-maximum suppression to six boxes by hand

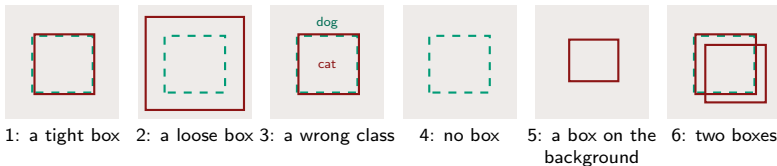
## Part 2 Metrics and data

Exercise: calculate the IoU of two box pairs

## Part 3 Deployment

Exercise: predict the frame rate when the resolution goes from 640 to 320

# Is this detection correct?



- ▶ Dashed and green: the label, which a person made. Solid and red: the prediction of the model.
- ▶ A classifier is correct or wrong for one image. A detector can make each of these errors for each object.
- ▶ To count the errors, you need three rules:
  - How much must the two boxes overlap?
  - Must the class be the same? Yes.
  - How many predictions can one label make correct? One.

**Key point:** All metrics of a detector start with one decision for each prediction: correct or not correct.

# IoU: the measure for the place of a box



IoU 0.9



IoU 0.7



IoU 0.5



IoU 0.3

- ▶ Part 1 used the IoU for two candidates. The metrics use it for a prediction and a label.
- ▶ A prediction is in the correct place when its IoU with the label is at or above a threshold.
- ▶ The usual threshold is 0.5. In the drawing, a box with the IoU 0.5 does not look good: one third of the label is not in the box.
- ▶ A second, more strict rule uses ten thresholds from 0.5 to 0.95.

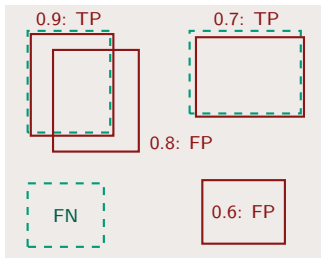
**Key point:** The IoU threshold of a metric is not the IoU threshold of the suppression. Name the threshold with each result.

| Side of the box | Error    | IoU  |
|-----------------|----------|------|
| 100 pixels      | 4 pixels | 0.92 |
| 40 pixels       | 4 pixels | 0.82 |
| 20 pixels       | 4 pixels | 0.67 |
| 10 pixels       | 4 pixels | 0.43 |

$$\text{IoU} = \frac{L - s}{L + s}$$

- ▶ A square box with the side  $L$  moves by  $s$  pixels to the right.
- ▶ The same error of 4 pixels is small for a large object. For an object of 10 pixels, the prediction is not correct at the threshold 0.5.
- ▶ The same rule applies to the label. A label that is 4 pixels wrong makes a correct prediction look wrong.

**Key point:** The metrics are strict for small objects. A larger input resolution gives each object more pixels (Part 3).



1. Sort the predictions by their score.
2. Take the next prediction. Find the label of the same class with the largest IoU that is still free.
3. If this IoU is at or above the threshold, the prediction is a **true positive** (TP). The label is not free now.
4. If not, the prediction is a **false positive** (FP).
5. A label with no prediction is a **false negative** (FN).

- ▶ A second box for one object is a false positive. A detector with no suppression gets a bad result.
- ▶ A detector has no true negatives: the number of places with no object has no limit. So nobody reports the accuracy of a detector.

## Precision

$$\frac{TP}{TP + FP} = \frac{713}{713 + 213} = 0.77$$

Of the boxes that the model gives, the part that is correct.

- ▶ The example: YOLO11n and the class “person” in 500 images, with the score threshold 0.25 and the IoU threshold 0.5. The images have 1069 labels of this class.
- ▶ The model gives 926 boxes. 713 are correct, and 213 are false alarms. It does not find 356 persons.
- ▶ Low precision: the product reports objects that are not there. Low recall: the product does not see objects.

Source: An experiment of this course with 500 images of COCO.

## Recall

$$\frac{TP}{TP + FN} = \frac{713}{713 + 356} = 0.67$$

Of the objects in the images, the part that the model finds.

# The score threshold: precision against recall

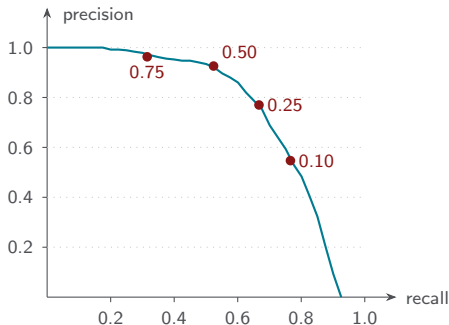
| Score threshold | TP  | FP  | FN  | Precision | Recall |
|-----------------|-----|-----|-----|-----------|--------|
| 0.10            | 819 | 677 | 250 | 0.55      | 0.77   |
| 0.25            | 713 | 213 | 356 | 0.77      | 0.67   |
| 0.50            | 560 | 45  | 509 | 0.93      | 0.52   |
| 0.75            | 337 | 13  | 732 | 0.96      | 0.32   |

- ▶ The same model, the same images, the class “person”. Only the score threshold changes.
- ▶ A higher threshold removes false positives: the precision goes up. It also removes true positives: the recall goes down.
- ▶ With the threshold 0.001, the model gives 13 032 boxes for 1069 persons. The recall is 0.90: no threshold finds each person.

**Key point:** One value of the precision or of the recall says nothing with no score threshold. Report the two values and the threshold together.

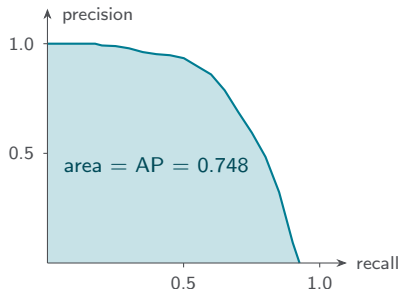
Source: An experiment of this course with 500 images of COCO.

# The precision-recall curve



- ▶ Each score threshold gives one point: a recall and a precision.
- ▶ All thresholds together give the curve.
- ▶ The red points are the four thresholds of the table.
- ▶ A better model has a curve nearer to the top right corner.
- ▶ You select the point of your product on this curve.

Source: YOLO11n, class "person", IoU threshold 0.5, 500 images of the validation set of COCO.  
An experiment of this course.

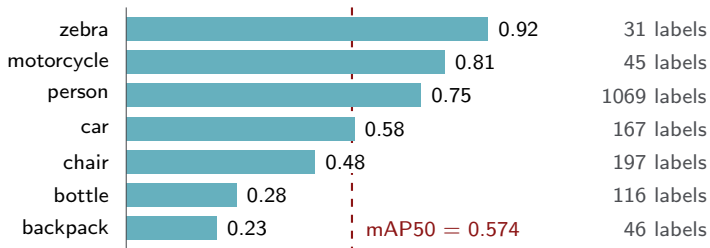


- ▶ AP: “average precision”. It is the area below the precision-recall curve of one class.
- ▶ It is the mean of the precision for all values of the recall.
- ▶ The AP uses no score threshold. It measures how well the scores sort the correct boxes before the false boxes.
- ▶ The largest value is 1.

**Key point:** The AP describes the model. The precision and the recall at one threshold describe the product.

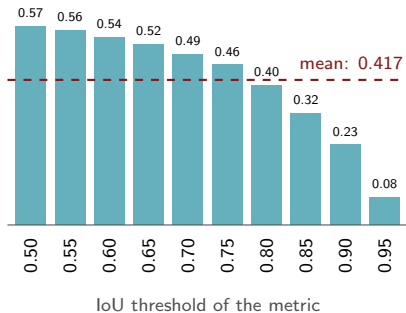
Source: The same curve as before. An experiment of this course.

# mAP: the mean over the classes



- ▶ mAP: “mean average precision”. It is the mean of the AP of all classes. The 500 images have labels of 78 classes.
- ▶ Each class has the same weight: the 31 zebras count as much as the 1069 persons.
- ▶ The mAP can hide a class that the model does not find. Read the AP of each class of your product.

Source: YOLO11n, AP at the IoU threshold 0.5 for 7 of the 78 classes. An experiment of this course.



- ▶ **mAP50:** the mAP with the IoU threshold 0.5. A loose box is correct.
- ▶ **mAP50-95:** the mean of the mAP for ten thresholds from 0.5 to 0.95. It is the main metric of the dataset COCO.
- ▶ A strict threshold gives a lower mAP: fewer boxes are exact enough.
- ▶ mAP50-95 is always the lower value. Here: 0.417 against 0.574.

**Key point:** mAP50 asks: does the model find the object? mAP50-95 asks also: is the box exact?

Source: An experiment of this course with 500 images of COCO.

```
yolo val model=yolo11n.pt data=val500.yaml imgsz=640
```

```
YOLO11n summary (fused): 100 layers, 2,616,248 parameters, 6.5 GFLOPs
```

| Class    | Images | Instances | Box(P | R     | mAP50 | mAP50-95) |
|----------|--------|-----------|-------|-------|-------|-----------|
| all      | 500    | 3504      | 0.674 | 0.53  | 0.574 | 0.417     |
| person   | 282    | 1069      | 0.781 | 0.655 | 0.748 | 0.515     |
| car      | 50     | 167       | 0.682 | 0.539 | 0.584 | 0.357     |
| backpack | 32     | 46        | 0.525 | 0.196 | 0.232 | 0.111     |

```
Speed: 0.7ms preprocess, 22.3ms inference, 0.0ms loss, 2.0ms postprocess
```

- ▶ Images: the images with a label of the class. Instances: the labels.
- ▶ P and R: the precision and the recall at the score threshold with the best balance of the two. They are not the values at your threshold.
- ▶ The run uses the score threshold 0.001, so that the curve is complete.
- ▶ The line Speed is the time for one image on the computer of the run. Here it is an x86 computer with 24 cores.

Source: An experiment of this course. The listing shows 4 of the 79 rows and a shorter first line.

# The result depends on the test images

| Images for the test of YOLO11n        | mAP50 | mAP50-95 |
|---------------------------------------|-------|----------|
| 128 images of its training set        | 0.670 | 0.502    |
| 500 images of the validation set      | 0.574 | 0.417    |
| All 5000 images of the validation set | 0.546 | 0.389    |

- ▶ The model saw the training images many times. A test with these images gives a result that is too good.
- ▶ 500 images are a sample. Their mAP50-95 is 0.028 above the value of all 5000 images.
- ▶ Ultralytics publishes 0.395 for this model on the 5000 images, with the tool of COCO. The validation of the package gives 0.389.

**Key point:** Test with images that the model did not see, from the camera and the place of the product. Report the number of images.

Source: An experiment of this course.  
Published value: the documentation of Ultralytics.

| Decision                                 | Number                                                              |
|------------------------------------------|---------------------------------------------------------------------|
| Compare two models, or float32 with int8 | mAP50-95 and mAP50 on the same test images                          |
| Set the score threshold of the product   | The precision and the recall at each threshold                      |
| A product that counts objects            | The precision and the recall together: each error changes the count |
| A product that must not miss an object   | The recall, with a limit for the false alarms each hour             |
| A product that sends alerts to a person  | The precision: too many false alerts, and nobody reads them         |
| Can the device run the model?            | The frames each second of the complete pipeline (Part 3)            |

**Key point:** The mAP selects the model. The precision and the recall at your threshold tell you what the user sees.

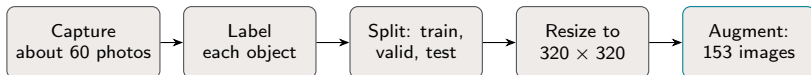
```
dataset/  
  data.yaml  
  train/images/img_001.jpg  
  train/labels/img_001.txt  
  valid/images/...  
  valid/labels/...  
  test/images/...  
  test/labels/...
```

```
# train/labels/img_001.txt  
0 0.512 0.431 0.220 0.305  
1 0.250 0.700 0.180 0.240  
  
# data.yaml  
train: train/images  
val: valid/images  
names: ['box', 'wheel']
```

- ▶ One text file for each image, with the same name. One line for each object: the class, then  $c_x$ ,  $c_y$ ,  $w$ ,  $h$  as parts of the image from 0 to 1.
- ▶ An image with no object has an empty file or no file. Such images teach the model the background.
- ▶ data.yaml names the folders and the classes. The class 0 is the first name.
- ▶ Three sets: train, validation for the decisions during the work, test for the final result.

Source: Format: kit lab "Object Detection".

# The dataset of the kit lab: box and wheel



- ▶ The goal: sort and count wheels and boxes. Two classes, and images with only the background.
- ▶ The photos come from the camera of the Raspberry Pi or from a phone, with different angles, backgrounds, and light.
- ▶ The augmentation makes new images from each photo: a rotation up to 15 degrees, a crop, a change of the brightness.
- ▶ Published result: mAP50 of 0.995 after 25 epochs with YOLOv8n.
- ▶ A test set from the same photo session is similar to the training set. The value says little about a different place.

**Key point:** Split first, then augment the training set only. Two copies of one photo in two sets make the test too easy.

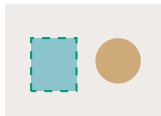
Source: Kit lab “Object Detection” of *Machine Learning Systems*.

| Question                             | Example of a rule                                       |
|--------------------------------------|---------------------------------------------------------|
| Which objects get a label?           | Each object of each class, in each image. No exception. |
| How tight is the box?                | The box touches the visible object on its four sides.   |
| An object that is partly hidden      | Label the visible part, if you can name the class.      |
| An object at the border of the image | Label it, if more than one half is in the image.        |
| A very small object                  | Label it from 10 pixels. Below that, no label.          |
| Two classes that look similar        | One sentence and two example images for each class.     |

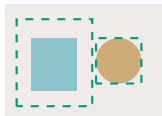
The rules are examples. Your product decides each rule. The same rules apply to the training set and to the test set.

**Key point:** Write one page of rules before the first label. Two persons with the rules must draw the same boxes.

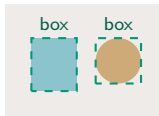
# Four types of label error



Missing label



Loose box



Wrong class



Extra label

| Error         | In the training set                                | In the test set                                  |
|---------------|----------------------------------------------------|--------------------------------------------------|
| Missing label | The model learns: this object is background.       | A correct prediction counts as a false positive. |
| Loose box     | The model learns loose boxes.                      | A correct prediction has a low IoU.              |
| Wrong class   | The model mixes two classes.                       | One false positive and one false negative.       |
| Extra label   | The model learns to see objects in the background. | A false negative that no model can avoid.        |

# Measured: label errors change the result

| Labels of the 500 test images    | Precision | Recall | mAP50 | mAP50-95 |
|----------------------------------|-----------|--------|-------|----------|
| The labels of COCO               | 0.674     | 0.530  | 0.574 | 0.417    |
| Sides moved by up to 5 percent   | 0.669     | 0.528  | 0.570 | 0.379    |
| Sides moved by up to 10 percent  | 0.666     | 0.522  | 0.562 | 0.315    |
| Sides moved by up to 20 percent  | 0.645     | 0.499  | 0.530 | 0.184    |
| 10 percent of the labels removed | 0.609     | 0.525  | 0.530 | 0.386    |
| 30 percent of the labels removed | 0.473     | 0.516  | 0.434 | 0.312    |
| 10 percent with a wrong class    | 0.618     | 0.414  | 0.425 | 0.310    |

- ▶ The model and the images are the same in each row. Only the labels change. A side moves by a random part of the size of its box.
- ▶ Loose boxes change mAP50 little. They cut mAP50-95 by more than one half.
- ▶ Missing labels cut the precision: the model finds objects that have no label.

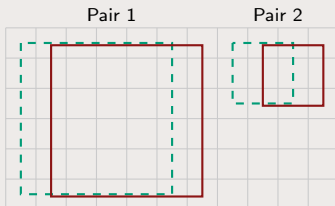
**Key point:** With bad test labels, you cannot measure a good model.

Source: An experiment of this course with YOLO11n.

| Check                                                                          | It finds                                                            |
|--------------------------------------------------------------------------------|---------------------------------------------------------------------|
| Draw the labels on the images.<br>Look at a sample of each class.              | Loose boxes, wrong classes                                          |
| Count the labels of each class,<br>and the images with no label.               | A class with too few examples, a typing<br>error in a class name    |
| Plot the widths and the heights<br>of all boxes.                               | Boxes with a size of zero, boxes as<br>large as the image           |
| Train a first model. Look at its<br>false positives with the highest<br>score. | Missing labels: the model is correct,<br>and the label is not there |
| Two persons label the same 20<br>images. Calculate the IoU of<br>their boxes.  | Rules that are not clear                                            |

**Key point:** The labels are a part of the model. Review them as you review code. After a correction, train again and measure again.

# Exercise: the IoU of two box pairs



|        | Label        | Prediction   |
|--------|--------------|--------------|
| Pair 1 | 0, 0, 10, 10 | 2, 0, 12, 10 |
| Pair 2 | 0, 0, 4, 4   | 2, 0, 6, 4   |

The numbers are  $x_1, y_1, x_2, y_2$  in pixels.  
The drawing shows the two pairs at two places, with the same scale.

1. Calculate the IoU of each pair.
2. Is each prediction a true positive with the IoU threshold 0.5? With the threshold 0.75?
3. The two predictions have the same error: 2 pixels to the right. Why are the two results different?
4. The model gives a second box for the label of pair 1: 1, 0, 11, 10, with a lower score. Is it a true positive with the threshold 0.5?

# Answer: the IoU of two box pairs

|        | Intersection       | Union                  | IoU  | At 0.5 | At 0.75 |
|--------|--------------------|------------------------|------|--------|---------|
| Pair 1 | $8 \times 10 = 80$ | $100 + 100 - 80 = 120$ | 0.67 | TP     | FP      |
| Pair 2 | $2 \times 4 = 8$   | $16 + 16 - 8 = 24$     | 0.33 | FP     | FP      |

1. The IoU is 0.67 and 0.33.
2. Pair 1 is a true positive at 0.5 only. Pair 2 is a false positive at the two thresholds. The label of pair 2 is then a false negative also: one error of the place gives two errors of the count.
3. The error is 20 percent of the side of the first box, and 50 percent of the side of the second box. The IoU measures the error as a part of the size of the object.
4. No. Its IoU is  $90/110 = 0.82$ , which is better than the first box. But the first box has the higher score and takes the label. The second box is a false positive. The suppression of Part 1 must remove it.

## Part 1 Detection models

Exercise: apply non-maximum suppression to six boxes by hand

## Part 2 Metrics and data

Exercise: calculate the IoU of two box pairs

## Part 3 Deployment

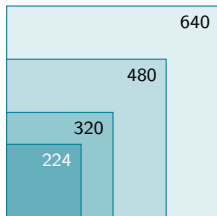
Exercise: predict the frame rate when the resolution goes from 640 to 320



- ▶ Part 2 gave the accuracy of a model. A product also needs a frame rate: the detections of each second.
- ▶ Four decisions set the frame rate. Each decision can cost accuracy.
- ▶ The syllabus of the lab asks for a table: two image sizes and two precisions, with the frame rate and the accuracy of each row.
- ▶ This part gives the method for each decision, with measured values of YOLO11n.

**Key point:** Report the frame rate and the accuracy together, for the file that runs on the board.

# The cost grows with the pixels

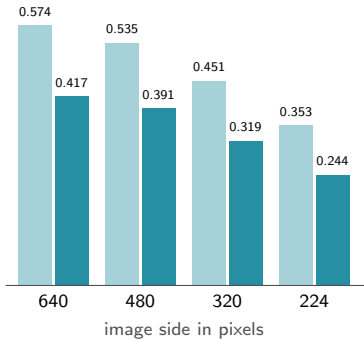


| Image side | Pixels      | Candidates | GFLOP |
|------------|-------------|------------|-------|
| 640        | 100 percent | 8400       | 6.5   |
| 480        | 56 percent  | 4725       | 3.7   |
| 320        | 25 percent  | 2100       | 1.6   |
| 224        | 12 percent  | 1029       | 0.8   |

- ▶ Each convolution does its work for each pixel of its map. One half of the image side gives one quarter of the operations.
- ▶ The parameters do not change: the file has the same size.
- ▶ Measured for the network in ONNX Runtime: 49.6 ms at 640 pixels and 13.6 ms at 320 pixels. The factor is 3.6.

**Key point:** The image size is the strongest control of the latency. It needs no new model.

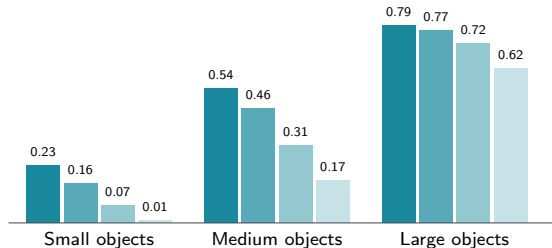
Source: An experiment of this course on an x86 processor, 4 threads.



- ▶ Light bars: mAP50. Dark bars: mAP50-95.
- ▶ The same weights, four image sizes. The model was trained at 640 pixels.
- ▶ From 640 to 480 pixels: 44 percent fewer operations, and mAP50-95 falls by 0.026.
- ▶ From 640 to 320 pixels: 75 percent fewer operations, and mAP50-95 falls by 0.099.

**Key point:** A smaller image is not free. Measure the mAP at each size that you want to use.

Source: YOLO11n on 500 images of COCO, an experiment of this course.



Recall at  
640, 480, 320,  
and 224 pixels,  
from dark to light

- ▶ Small: a label with an area below  $32 \times 32$  pixels in the photo. Large: above  $96 \times 96$  pixels.
- ▶ At 320 pixels, the model finds 7 percent of the small objects. It still finds 72 percent of the large objects.
- ▶ An object of 32 pixels covers 4 cells on a side of the finest map. At one half of the image size, it covers 2 cells.

**Key point:** Ask how many pixels the smallest object of your product has. This number selects the image size.

Source: YOLO11n on 500 images of COCO: score threshold 0.25, IoU threshold 0.5. An experiment of this course.

# The model scale: n, s, m, l, x

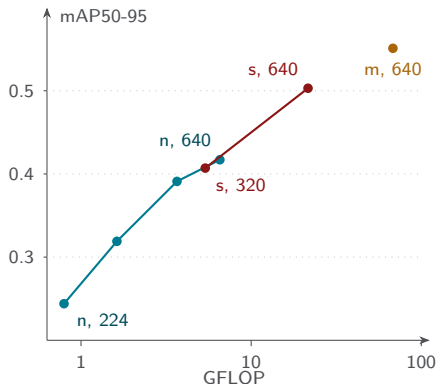
| Model   | Parameters   | GFLOP | Published | 500 images   |
|---------|--------------|-------|-----------|--------------|
| YOLO11n | 2.6 million  | 6.5   | 0.395     | 0.417        |
| YOLO11s | 9.4 million  | 21.6  | 0.470     | 0.503        |
| YOLO11m | 20.1 million | 68.1  | 0.515     | 0.551        |
| YOLO11l | 25.3 million | 87.2  | 0.534     | not measured |
| YOLO11x | 56.9 million | 195.3 | 0.547     | not measured |

- ▶ The two last columns give mAP50-95 at 640 pixels. A scale changes the number of channels and the number of layers.
- ▶ From n to s: 3.3 times the operations for 0.086 more mAP50-95. From s to m: 3.2 times the operations for 0.047 more.
- ▶ The gain of each step gets smaller. The cost of each step does not.
- ▶ The Raspberry Pi 5 has no accelerator. The lab uses the scale n.

**Key point:** Start with the smallest scale. Go up only when the accuracy is not sufficient and the frame rate has a margin.

Source: Published: the documentation of Ultralytics, 5000 images of COCO. Measured: an experiment of this course.

# A larger model, or a larger image?

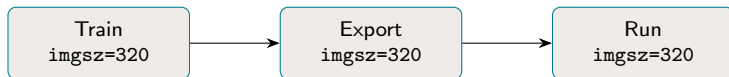


- ▶ The horizontal axis has a logarithmic scale.
- ▶ YOLO11n at 640 pixels: 6.5 GFLOP and 0.417.
- ▶ YOLO11s at 320 pixels: 5.4 GFLOP and 0.407.
- ▶ The two choices cost almost the same, and they give almost the same mAP.
- ▶ The small image of YOLO11s loses the small objects. Its file is 3.4 times as large.

**Key point:** The mAP does not decide here. The size of your objects and the memory of the device decide.

Source: 500 images of COCO, an experiment of this course.

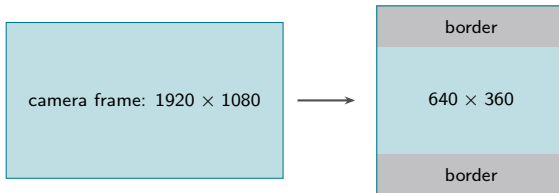
# The image size rule: train, export, run



- ▶ The PyTorch file accepts each image size. An exported file has a static input shape (Day 7): the size is a part of the file.
- ▶ The export uses 640 pixels when you give no size.
- ▶ Published case: a model that was trained at 320 pixels and exported with no size gave 113 boxes and 164 wheels for an image with one box. The program gave no error and no warning.
- ▶ A model that was trained at 320 pixels saw objects of a certain size in pixels. At a different size, the objects look different to it.

**Key point:** Use one image size for the training, the export, and the inference. Check the shape in the first output line.

Source: *Edge AI Engineering*, "Computer Vision Applications with YOLO".



- ▶ The program makes the frame smaller with one factor for the two sides, and grey borders fill the square. After the inference, it moves each box back to the frame.
- ▶ For a frame of 16:9, the borders are 44 percent of the square. The network does this work for nothing.
- ▶ An export with `imgsz=384,640` gives 5040 candidates, not 8400. Measured in ONNX Runtime: 29.9 ms, not 49.6 ms.

**Key point:** The pre-processing of the device must be the pre-processing of the training. A different resize changes the accuracy.

Source: An experiment of this course on an x86 processor.

| Format       | Files               | Size     | Export | Runtime      |
|--------------|---------------------|----------|--------|--------------|
| PyTorch      | yolo11n.pt          | 5.35 MB  | none   | PyTorch      |
| ONNX         | yolo11n.onnx        | 10.24 MB | 5 s    | ONNX Runtime |
| NCNN         | .param and .bin     | 10.10 MB | 8 s    | NCNN         |
| LiteRT       | yolo11n.tflite      | 10.27 MB | 19 s   | LiteRT       |
| LiteRT, int8 | yolo11n_int8.tflite | 2.94 MB  | 136 s  | LiteRT       |

- ▶ The PyTorch file stores the weights in float16. The three exports store float32: the files are two times as large.
- ▶ The int8 export needs calibration images. It ran the model on 200 images.
- ▶ The three runtimes are the runtimes of Day 7. The export fuses each batch normalization into its convolution.

Source: YOLO11n at 640 pixels, an experiment of this course. 1 MB is 1024 KB.

```
# On the laptop: export the trained model for the two runtimes
yolo export model=best.pt format=ncnn imgsz=320
yolo export model=best.pt format=litert imgsz=320 quantize=8 data=data.yaml

# On the Raspberry Pi: run, then measure
yolo predict model=best_ncnn_model imgsz=320 source=test.jpg
yolo val model=best_int8.tflite imgsz=320 data=data.yaml
```

- ▶ `best.pt` is the file of your training. The export writes the folder `best_ncnn_model` and the file `best_int8.tflite`.
- ▶ `quantize=8` makes a model with int8 weights and int8 activations. `data` names the dataset: the export takes the calibration images from its validation set.
- ▶ The export to LiteRT runs on a laptop with Linux or macOS. It does not run on the Raspberry Pi.
- ▶ `predict` and `val` accept an exported file. The package selects the runtime from the name.

Source: The Ultralytics package, version 8.4.171. The export commands ran with `yolo11n.pt` at 640 pixels.

# Measured: one model in four files

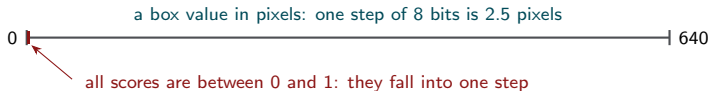
| File and runtime     | mAP50 | mAP50-95 | Network, 4 threads |
|----------------------|-------|----------|--------------------|
| PyTorch              | 0.579 | 0.424    | 69.1 ms            |
| ONNX in ONNX Runtime | 0.579 | 0.424    | 49.6 ms            |
| NCNN                 | 0.579 | 0.424    | 60.9 ms            |
| LiteRT, float32      | 0.579 | 0.424    | 34.8 ms            |

- ▶ The four files give the same mAP. An export in float32 does not change the accuracy.
- ▶ The times are from an x86 processor. NCNN is made for Arm: on a Raspberry Pi, the order can change.
- ▶ This test puts each image into a square of 640 pixels. With the rectangle of Part 2, the same model has 0.417. The pre-processing is a part of the result.

**Key point:** After each export, run the validation again with the exported file. Equal values are the proof.

Source: YOLO11n on 500 images of COCO, an experiment of this course on an x86 processor. The times say nothing about a board.

# int8 for a detector: boxes and scores



- ▶ The output tensor of YOLO holds box values from 0 to 640 and scores from 0 to 1. A tensor in `int8` has one scale (Day 4).
- ▶ One scale for 640 gives steps of 2.5. Each score becomes 0: the model finds nothing.
- ▶ The export of the package divides the box values by the image size. All values are then between 0 and 1.
- ▶ In the `int8` file of the experiment, the scale of the output is 0.0039. One step of a box value is 2.5 pixels at 640 pixels.
- ▶ The input and the output of the file stay in `float32`. The file converts at its two ends.

**Key point:** Look at the range of each output before you quantize a detector. Two ranges in one tensor need a change of the model.

Source: The export code of the Ultralytics package, and the exported file read with LiteRT. An experiment of this course.

| LiteRT file                     | Size     | mAP50 | mAP50-95 | Network      |
|---------------------------------|----------|-------|----------|--------------|
| float32                         | 10.27 MB | 0.579 | 0.424    | 34.8 ms      |
| int8 weights only               | 2.90 MB  | 0.580 | 0.419    | 53.7 ms      |
| int8, calibration: 200 photos   | 2.94 MB  | 0.517 | 0.331    | 55.5 ms      |
| int8, calibration: noise        | 2.94 MB  | 0.493 | 0.299    | not measured |
| int8 weights, int16 activations | 3.00 MB  | 0.573 | 0.415    | 2329 ms      |

- ▶ int8 weights alone cost 0.005 of mAP50-95. Full int8 costs 0.062 of mAP50 and 0.093 of mAP50-95: the boxes lose more than the classes.
- ▶ Calibration with noise costs 0.032 more of mAP50-95. Use photos of your application, as on Day 4.
- ▶ With int16 activations, the loss is 0.009 of mAP50-95. But LiteRT has no fast kernels for them on this processor: the network needs 67 times the time of float32.
- ▶ On this x86 processor, int8 is slower than float32. For a classifier on the Raspberry Pi 5, a published test gave int8 a factor of 2.9 (Day 7).

Source: An experiment of this course on an x86 processor.

| Model                                 | Board                 | PyTorch      | NCNN        |
|---------------------------------------|-----------------------|--------------|-------------|
| YOLO11n for box and wheel, 320 pixels | Raspberry Pi 5        | about 400 ms | about 80 ms |
| YOLO11n for COCO, 640 pixels          | Raspberry Pi Zero 2 W | 4672 ms      | 2748 ms     |

- ▶ The times are for the network alone.
- ▶ On the Raspberry Pi 5, the export to NCNN gives a factor of 5. It costs no accuracy.
- ▶ The SSD MobileNet V1 file of Part 1 needs about 100 ms on a Raspberry Pi 5 in LiteRT.
- ▶ The sources give no time for a YOLO model in LiteRT with `int8` on the Raspberry Pi 5. You measure it in the lab.

**Key point:** First export to a runtime for the board. Then measure.

Source: *Edge AI Engineering*, chapters on detection and YOLO.

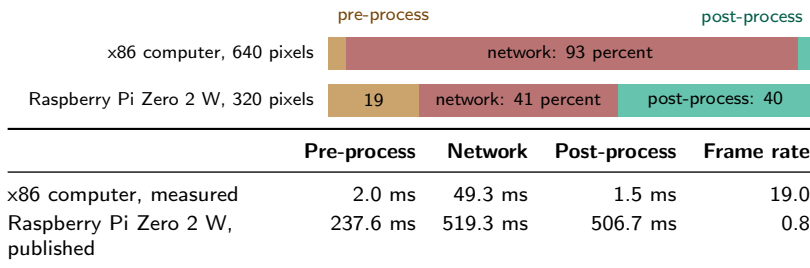
# The pipeline of one frame



| Step           | Work                                                           | Who does it                   |
|----------------|----------------------------------------------------------------|-------------------------------|
| Capture        | Wait for the next frame of the camera. Copy it to the program. | The camera and its driver     |
| Pre-process    | Letterbox, colour order, float32, values from 0 to 1           | OpenCV and numpy on the CPU   |
| Infer          | The network                                                    | The runtime, with its threads |
| Post-process   | Score threshold, suppression, boxes back to the frame          | Python code on one core       |
| Use the result | Draw, count, send a message (Week 3)                           | Your program                  |

**Key point:** The frame rate is one divided by the sum of all steps, not by the time of the network.

# The time of each step: two devices



- ▶ The frame rate is in frames each second. On the small board, the network alone would give 1.9.
- ▶ The pre-processing and the post-processing are Python code. They get no gain from a faster runtime or from int8.
- ▶ Published for the same board: 2545 ms for the network in the first run, and 519 ms in the second run. Do not count the first run.

Source: x86: an experiment of this course, YOLO11n in ONNX Runtime. Raspberry Pi: *Edge AI Engineering*, chapter on YOLO.

| Limit                | What happens                                                                             |
|----------------------|------------------------------------------------------------------------------------------|
| The camera           | A camera with 30 frames each second gives one frame each 33 ms. A faster pipeline waits. |
| The sum of the steps | A pipeline of 100 ms gives 10 results each second.                                       |
| Old frames           | The driver keeps frames in a queue. A slow program reads frames that are old.            |
| The cores            | Four threads for the network leave no core for the capture and for Python (Day 7).       |
| The heat             | At full load, the processor can reduce its clock after some minutes (Day 9).             |

- ▶ Read the camera in a separate thread, and keep only the newest frame.
- ▶ The frame rate says how many results you get. The latency says how old a result is.

**Key point:** Measure the frame rate over one minute of the real pipeline.

# How to reach a target frame rate

| Action                                 | Cost                                                                    |
|----------------------------------------|-------------------------------------------------------------------------|
| 1 Export to a runtime for the board    | None. The mAP stays the same.                                           |
| 2 Use a rectangle for a frame of 16:9  | None. The model does not see the borders.                               |
| 3 Use a smaller image                  | The small objects. Measure the mAP again.                               |
| 4 Use int8                             | Some mAP, mostly at the strict thresholds. It needs calibration photos. |
| 5 Use a smaller scale                  | Accuracy, and the time for a new training                               |
| 6 Run the network on each second frame | A moving object is found later.                                         |

- ▶ After each action, measure the frame rate of the pipeline on the board and the mAP of the file. Stop when the target is reached.

**Key point:** Take the actions in this order: the first actions cost nothing. Write each step in the Decision Log.

| Part | Topic              | Work                                                                                                     |
|------|--------------------|----------------------------------------------------------------------------------------------------------|
| A    | Pre-trained models | Run the SSD model and a YOLO model on test images on the Raspberry Pi.                                   |
| B    | Custom model       | Train a YOLO model on a small dataset with two classes, on the laptop or on Colab.                       |
| C    | Export and deploy  | Export to NCNN and to LiteRT in <code>int8</code> .<br>Run the live detection with the camera.           |
| D    | Measure            | The frame rate for two image sizes and two precisions. The accuracy loss of the <code>int8</code> model. |

- The check: the model detects the two classes in the live image, and the table has four measured rows.

**Key point:** Write your prediction for each row before Part D. Measure these four rows on a Raspberry Pi 5 in the lab.

# Exercise: the frame rate from 640 to 320

A Raspberry Pi 5 runs a detector on the frames of its camera. The table gives example values for an image size of 640 pixels.

|                            |        |
|----------------------------|--------|
| Capture and pre-processing | 15 ms  |
| Network                    | 300 ms |
| Post-processing            | 10 ms  |

1. Calculate the frame rate at 640 pixels.
2. You export the model for 320 pixels. Predict the time of each step and the new frame rate.
3. By which factor does the frame rate change? Why is the factor not 4?
4. An object has  $40 \times 40$  pixels in the image of 640 pixels. Which size does it have at 320 pixels? What do you expect for its recall?
5. The target is 15 frames each second. Name the next action.

1.  $15 + 300 + 10 = 325$  ms. The frame rate is  $1000/325 = 3.1$  frames each second.
2. The network has one quarter of the pixels:  $300/4 = 75$  ms. The capture and the post-processing change little: 15 ms and 10 ms. The sum is 100 ms: 10 frames each second.
3. The factor is  $325/100 = 3.25$ . The 25 ms outside the network do not change. This is the law of Amdahl of Day 7.
4.  $20 \times 20$  pixels. The object changes from “medium” to “small”. In the experiment, the recall of small objects is far below the recall of medium objects: expect a large loss.
5. 15 frames each second need 67 ms. The network must go from 75 ms to about 42 ms. Try `int8`, and measure the mAP. A rectangle for a frame of 16:9 costs no accuracy: try it first.

Measured on the x86 computer: the network went from 49.3 ms to 13.6 ms (a factor of 3.6), and the pipeline from 52.8 ms to 15.7 ms (a factor of 3.4).

- ▶ A single-stage detector predicts all candidate boxes in one pass of the network. Non-maximum suppression keeps the best box of each object.
- ▶ IoU measures the overlap of two boxes. Precision, recall, and mAP come from the IoU matches between the predicted boxes and the labels.
- ▶ The labels set the limit of the model. A model learns the errors of its labels, so one labelling rule applies to all images.
- ▶ The input resolution, the model scale, and the precision set the latency of the model. Capture, pre-processing, and post-processing add to it.

**Key point:** A detector on an edge device is a pipeline, not only a model. Measure the frame rate of the complete pipeline, and report the mAP of the model file that you deploy.

## Book chapters

- ▶ *Machine Learning Systems* (mlsysbook.ai): Raspberry Pi lab “Object Detection”, and Volume I, chapter 10 “Model Compression” (calibration).
- ▶ *Edge AI Engineering*: “Object Detection: Fundamentals”, “Computer Vision Applications with YOLO”, and “Custom Object Detection Project”.

## Papers

- ▶ “You Only Look Once: Unified, Real-Time Object Detection”, Redmon, Divvala, Girshick, and Farhadi, 2016.
- ▶ “SSD: Single Shot MultiBox Detector”, Liu et al., 2016.
- ▶ “EfficientDet: Scalable and Efficient Object Detection”, Tan, Pang, and Le, 2020.

## Tools and data

- ▶ Ultralytics YOLO: <https://docs.ultralytics.com>
- ▶ Raspberry Pi guide: <https://docs.ultralytics.com/guides/raspberry-pi>
- ▶ COCO dataset: <https://cocodataset.org>

- ▶ *Machine Learning Systems* by Vijay Janapa Reddi and contributors, Harvard University. [mlsysbook.ai](https://mlsysbook.ai), CC BY-NC-SA 4.0.
- ▶ *Edge AI Engineering: Raspberry Pi* by Marcelo Rovai. [github.com/Mjrovai/EdgeML\\_Made\\_Ease\\_ebook](https://github.com/Mjrovai/EdgeML_Made_Ease_ebook) and [github.com/Mjrovai/EdgeML-with-Raspberry-Pi](https://github.com/Mjrovai/EdgeML-with-Raspberry-Pi), GPL-3.0.